

Lab 1: Looking Under the Hood of R

Predict, then run. The point is to catch R in the act.

Work in an **R script**, not only in the console. Save it as `lab01.R`.

Do **1.1**, **1.2**, **1.3**, and **1.4** first. You will need `tidyverse` installed as in note 1.2, and you will import `students.csv` as in note 1.3. Note **1.5** is not needed for this lab.

Download [students.csv](#) and put it in your working directory (`getwd()` should see it), or use a relative path from note 1.3.

For every item marked **Predict**, do the following in order:

1. Write your prediction in a comment, *before* you run the code.
2. Run the code.
3. Write a short explanation of *why* R did that, using the rules from the notes. "R printed that" is not an explanation.

The notes give you the rules. The lab asks you to apply them *before* you see the output. Some results will surprise you. That is the point. You should not need material that is not in 1.1–1.4.

Do not look up the answer until you have a written prediction.

A. What `c()` actually builds

R's atomic vectors are not "lists of values of any type." They are typed objects. `c()` is willing to rewrite your values in order to keep that true.

A1. Predict

```
typeof(c(1, TRUE))
typeof(c(1L, TRUE))
typeof(c(1, "2", TRUE))
```

Why are the three answers not the same? What did `TRUE` become in each case?

A2. Predict

```
TRUE + TRUE + TRUE
c(TRUE, FALSE, TRUE) * 2
```

If `TRUE` is a logical value, why is arithmetic legal?

A3. Predict

```
1 == 1L
typeof(1)
typeof(1L)
is.numeric(1L)
```

1 and 1L compare as equal. Are they the same type? What does `is.numeric()` say about an integer?

A4. Predict

```
f <- factor(c("10", "2", "2"))
as.numeric(f)
as.numeric(as.character(f))
```

Note 1.4 treats factors as categorical data with levels. What is `as.numeric()` looking at when the object is a factor?

B. Three kinds of nothing

NA, NaN, and NULL are not three names for the same idea.

B1. Predict

```
x <- c(NA, NaN, Inf, NULL)
x
length(x)
typeof(x)
```

You passed four things to `c()`. Why does `x` not have length 4?

B2. Predict

```
NA == NA
NaN == NaN
NULL == NULL
```

Then:

```
is.na(NA)
is.nan(NaN)
is.null(NULL)
```

Why is `==` a poor way to test for these values? Which `is.*()` function matches which kind of nothing?

B3. Predict

```
is.na(NaN)
is.nan(NA)
typeof(NA)
typeof(c(NA, 1))
typeof(c(NA, "a"))
```

NA looks like one object. Use the coercion hierarchy from note 1.4. What happened to NA when it entered those two vectors?

C. Indexing is a language

[is not "get the nth thing." It is a function with rules for integers, negatives, logicals, and names.

C1. Predict

```
x <- 10:13
x[c(1, 1, 1)]
x[-c(1, 1)]
```

Positive indices select. Negative indices drop. Why does repeating 1 mean two different things in these two lines?

C2. Predict

```
x <- c(10, 20, 30, 40)
x[c(TRUE, FALSE)]
x[c(TRUE, FALSE, TRUE)]
```

Note 1.4 says R recycles the shorter vector in arithmetic. What rule is R using here, and where does the extra TRUE go?

C3. Predict

```
student <- list(name = "Ada", age = 36, grades = c(90, 85))
student[1]
student[[1]]
student$name
student["name"]
```

Note 1.4 distinguishes [, [[, and \$. What does each line return, and which results are still lists?

C4. Predict

```
students <- data.frame(name = c("Ada", "Bob"), grade = c(91, 74))  
class(students[, "grade"])  
class(students[, "grade", drop = FALSE])  
class(students["grade"])  
class(students[["grade"]])
```

One of these is a data frame, one is a vector, and the difference is easy to miss in a larger script. Which is which, and what is `drop` for?

C5. Predict

```
m <- matrix(1:6, nrow = 2)  
m  
m[2, 1]
```

Fill in the matrix on paper before you run this, using `matrix(1:6, nrow = 2)` from note 1.4. In what order does R fill the matrix, and why is `m[2, 1]` equal to 2 rather than 4?

D. Packages, names, and what R is searching

A function name is not a unique identity. It is a name R looks up on a search path.

D1. Explain, then try only the quoted form for installation

Why does this fail (do not keep running it):

```
install.packages(ggplot2)
```

while this is the intended form:

```
install.packages("ggplot2")
```

and yet both of these can succeed:

```
library(ggplot2)  
library("ggplot2")
```

What is `library()` doing with an unquoted name that `install.packages()` will not do?

D2. Predict, in a session where you have not yet loaded dplyr

```
students <- data.frame(name = c("Ada", "Bob"), grade = c(91, 74))
filter
filter(students, grade > 80)
```

Then load dplyr and run the same two lines again.

```
library(dplyr)
filter
filter(students, grade > 80)
```

Note 1.2 covers `library()`, `search()`, and `dplyr::filter()`. There is already a `filter()` in Base R / stats. What changed after `library(dplyr)`? How can two functions share a name? How do you call the older one anyway?

D3. Predict

```
x <- c(10, 20, 30)
x |> mean()
x %>% mean()
```

Run this *before* loading tidyverse. Why does one pipe exist in Base R and the other does not? After `library(tidyverse)`, run both lines again.

D4. Predict

```
search()
library(ggplot2)
search()
```

Where did `ggplot2` go on the search path, and why does that position matter when two packages contain the same function name?

E. Arithmetic that is not always the arithmetic you expect

E1. Predict

```
c(1, 2) + c(10, 20, 30, 40)
1:3 * 1:6
```

Note 1.4 says R recycles the shorter vector. Write the recycled vectors out by hand. When does R warn, and when does it stay silent?

E2. Predict

```
1 / 0
0 / 0
TRUE / FALSE
Inf - Inf
NA + 1
NULL + 1
```

Which of these are `Inf`, `NaN`, `NA`, and which is an error? What coercion makes `TRUE / FALSE` legal?

F. Files, data frames, and shape

Place `students.csv` in your working directory.

F1. Run and compare

```
base_students <- read.csv("students.csv")
tidy_students <- readr::read_csv("students.csv")

class(base_students)
class(tidy_students)
str(base_students)
str(tidy_students)
```

Both read the same file. Why are the objects not the same class? What extra classes does `read_csv()` attach, and what is a tibble claiming to be besides a data frame?

F2. Predict

```
base_students$grade * 0.01
base_students[, "grade"]
base_students["grade"]
```

One of these uses the vectorized arithmetic from note 1.4. Which extraction still has two dimensions?

What to submit

Submit `lab01.R` with:

- a prediction comment above every **Predict** block
- the code
- a short explanation comment below the output of each block

If a result still feels impossible after you have explained it, write down the question you now have. That question is part of the lab.

Solutions are on a [separate page](#) and open only with a password.